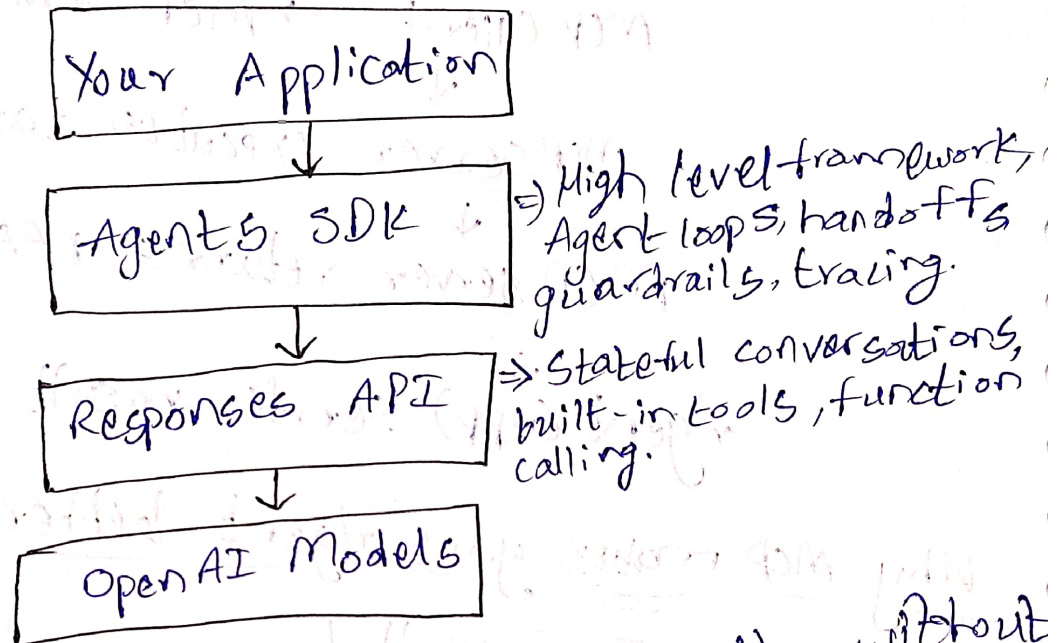


OpenAI Agent Stack

AI Agent (LLM-based) = LLM + Tools + Loop.

OpenAI Models

→ GPT 5.5, 5, 4.1 ⇒ reasoning & non-reasoning.



*We can use responses-api directly without to SDK.

* Agents SDK handles the hard parts, no need to build complete framework, but whereas in using the responses-API directly you have to build everything from the scratch.

→ Simply, with Responses API, you build the loop. With Agents SDK, the loop is built for you.

Responses API (Open AI)

→ The core interface to interact with Open AI models. Allows you to send input, receive structured output, enable tool usage & chain conversations across turns.

Features

→ Simpler Input

* Pass a string or structured input.

→ Conversation Chaining (Best feature of Responses API).

* You can control when conversations are connected.

→ Tool Integration

* Enables tools like web search or code execution.

→ Model-Driven Tool use

* Model decides when to call tools.

→ Efficient Execution

* Designed for improved performance & caching.

Built-in Tools

1) Web Search ⇒ { "type": "web-search" }
* Search for the current info from internet.

2) File Search ⇒ { "type": "file-search" }
* Search through uploaded documents.

3) Code Interpreter ⇒ { "type": "code-interpreter" }
* Write & execute python code.

4.) Computer Use = {"type": "computer-use-preview"}

* Control a computer interface.

5.) Image Generation = {"type": "image-generation"}

* Generate images via DALL-E.

Agents SDK (Open AI)

* A Python framework that helps you define agents, manage tool usage & build multi-agent systems.

* Built on top of the Responses API.

Components

→ Agent

→ Runner = Executes the agent loop.

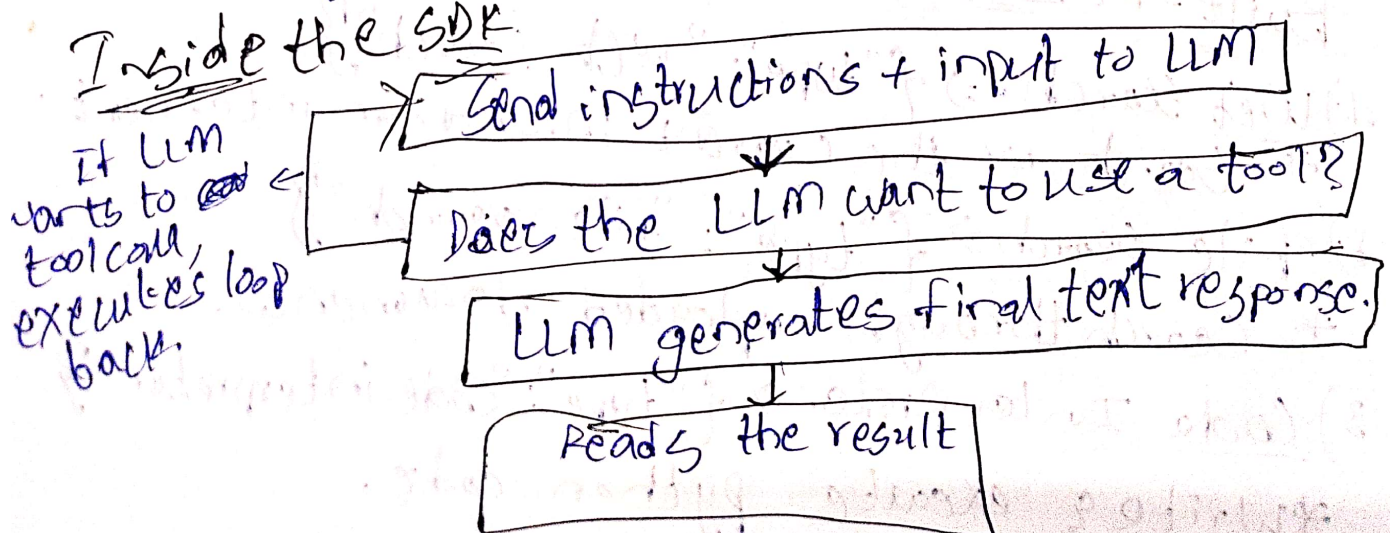
→ Tools = Functions the agent can call.

→ Handoffs = Transfer control b/w agents.

→ Guardrails = Validate inputs & outputs for safety.

→ Tracing = Monitor what the agent did.

Inside the SDK



Tools & Function Calling

Tools (Gives Agents super powers).

→ Hosted Tools ⇒ Provide by OpenAI.

* Mostly Built-in Tools.

→ Function Tools

→ Agents as Tools ⇒ one agent can be used by other.

SDK vs LangChain

SDK	LangChain
→ init Model is set inside the agent	→ Have to initialize the chat model.
→ @function = tool	→ @tool.
→ But in SDK it is done in complete step.	→ While creating the agent, LangChain separates the model, tools & orchestration into multiple layers.
→ Runner.run-sync (This runs the agent)	→ agent.invoke (This runs the agent).
→ result.final_output (Reads the answer)	→ Parse result [] (Reads the answer).

Multi-Agent Systems

→ Handoffs (Decentralized).

- * Agents transfer control to each other.
- * The receiving agent takes over the conversation completely.
- * This is best for routing, customer support & task delegation.

→ Manager

- * A central agent calls specialized agents as tools.
- * The manager keeps control of the conversation.



- * This is best for complex orchestration & unified experience.

Guardrails & Safety

Guardrails ⇒ Keeping Agents Safe

Input Guardrails

- * Run before the agent processes input.

uses

- Blocks off-topic requests.
- Detect prompt injection attempts.
- Validate input format is in expected format.

Output Guardrails

* Run After the agent generates output.

Uses

- * Checks for hallucinations.
- * Enforce brand guidelines & tone.
- * Prevent sensitive data leakage.

Working

- * Guardrail is like a small helper agent (or) function that checks input/output.
- * If the check fails, it triggers a 'tripwire' - an exception that stops the main agent before any damage is done.

Tracing

- * The ability to see what the agent did